Data Structures: Assignment-11

Name: Suraj Shah

Admission Number: I24AI012

1. Find the kth smallest element in an array using quicksort.

Code:

```
#include <iostream>
using namespace std;

int partition(int a[], int low, int high) {
    int pivot=a[high];
    int i=low-1;
    for (int j=low; j<high; j++) {
        if (a[j]<pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i+1], a[high]);
    return i+1;
}

int quickSort(int a[], int low, int high, int k) {
    if (low<high) {
        int pivot=partition(a, low, high);

        if (pivot==k) {
            return a[pivot];
        }

        else if (pivot>k) {
            return quickSort(a, low, pivot-1, k);
        }

        else {
            quickSort(a, pivot+1, high, k);
        }
    }
}

void printArray(int arr[], int n) {
    for (int i=0; i<n; i++) {
        cout << arr[i] << " ";
```

```
    }
    cout << endl;
}

int main() {
    int n, k;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
    for (int i=0; i<n; i++) {
        cin >> arr[i];
    }
    cout << "Array: ";
    printArray(arr, n);

    cout << "Enter the value of k: ";
    cin >> k;

    cout << "The kth smallest element is: " << quickSort(arr, 0, n-1, k-1);
}
```

Output:

```
Enter the number of elements: 6
Enter the elements: 54 2 38 4 99 1
Array: 54 2 38 4 99 1
Enter the value of k: 3
The kth smallest element is: 4
```

2. Given an array of non-negative integers, implement the Counting Sort algorithm to sort the array in ascending order.

Code:

```
#include <iostream>
#include <algorithm>
using namespace std;

void countingSort(int arr[], int n) {
    int max=*max_element(arr, arr+n);

    int count[max+1]={0};
    for (int i=0; i<n; i++) {
        count[arr[i]]++;
    }

    int index = 0;
    for (int i = 0; i <= max; i++) {
        while (count[i] > 0) {
            arr[index++] = i;
            count[i]--;
        }
    }
}

void printArray(int arr[], int n) {
    for (int i=0; i<n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
    for (int i=0; i<n; i++) {
        cin >> arr[i];
    }
    cout << "Array: ";
    printArray(arr, n);

    countingSort(arr, n);

    cout << "Sorted Array using Counting Sort: ";
```

```
    printArray(arr, n);
}
```

Output:

```
Enter the number of elements: 5
Enter the elements: 2 8 1 6 4
Array: 2 8 1 6 4
Sorted Array using Counting Sort: 1 2 4 6 8
```

3. Given an array of non-negative integers, implement the Radix Sort

algorithm to sort the array in ascending order.

Code:

```
#include <iostream>
#include <algorithm>
using namespace std;

void countingSort(int arr[], int size, int exp) {
    const int base = 10;
    int output[size];
    int count[base] = {0};

    for (int i = 0; i < size; i++) {
        int digit = (arr[i] / exp) % base;
        count[digit]++;
    }

    for (int i = 1; i < base; i++) {
        count[i] += count[i - 1];
    }

    for (int i = size - 1; i >= 0; i--) {
        int digit = (arr[i] / exp) % base;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    for (int i = 0; i < size; i++) {
        arr[i] = output[i];
    }
}

void radixSort(int arr[], int size) {
    int maxVal = *max_element(arr, arr + size);

    for (int exp = 1; maxVal / exp > 0; exp *= 10) {
        countingSort(arr, size, exp);
    }
}

int main() {
    int arr[] = {170, 45, 75, 90, 802, 24, 2, 66};
    int size = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
```

```
    cout << endl;

    radixSort(arr, size);

    cout << "Sorted array: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

Output:

```
Enter the number of elements: 8
Enter the elements: 170 45 75 90 802 24 2 66
Original array: 170 45 75 90 802 24 2 66
Sorted array: 2 24 45 66 75 90 170 802
```